

AIG - Configuration API Design Draft

Background

AIG configuration today requires SSH + interactive TUI. This works for manual operator workflows but cannot be automated or integrated into CI/CD pipelines or external tooling.

The **Configuration API** is a new HTTP service that exposes configuration operations programmatically. Phase 1 covers enrollment and certificate import.

Authentication

Operator generates a token via SSH CLI. SSH is the existing trust boundary — no new network-facing auth endpoint is introduced.

Enabling the API and getting a token:

```
1 ssh nsadmin@<aig-ip> enable-config-api --ttl 7d          # default
  port 8443
2 ssh nsadmin@<aig-ip> enable-config-api --port 9443 --ttl 7d # custom
  port
3
4 # Output:
5 {
6   "token": "3f2a1b8ec8d...e8c4d9.1749945600",
7   "expires_at": "2026-06-10T10:00:00Z"
8 }
```

The token is printed once and never retrievable again. Re-running the command generates a new token and immediately invalidates the previous one.

Token format:

```
1 <random_32_bytes_hex>.<unix_timestamp>
```

The expiry timestamp is embedded in the token string. AIG stores only:

```
1 token_hash = sha256(full_token_string)
```

Verification on each request:

```
1 1. Split token by "." → extract unix_timestamp
2 2. Check now < unix_timestamp (not expired)
3 3. Check sha256(incoming_token) == stored_hash (not tampered)
```

Usage:

```
1 Authorization: Bearer <token>
```

API Endpoints

All endpoints require HTTPS

PUT /v1/enrollment

Submit an enrollment token to enroll the AIG appliance. The tenant FQDN is extracted from the token itself.

Example:

```
1 curl -X PUT https://<aig-ip>:8443/v1/enrollment \  
2   -H "Authorization: Bearer 3f2a1b8ec8d...e8c4d9.1749945600" \  
3   -H "Content-Type: application/json" \  
4   -d '{"token": "<enrollment_token>"}'
```

Response 200:

```
1 {  
2   "status": "in_progress",  
3   "gateway_id": "7708863f-2bb4-42dd-9228-ba41bf8aaa10"  
4 }
```

Enrollment is asynchronous — poll `GET /v1/enrollment/status` to check when it completes.

Status values: `awaiting_submission` / `in_progress` / `completed` / `failed` / `unknown`

Error responses:

- `409` — already enrolled

GET /v1/enrollment/status

Returns the current enrollment state.

Example:

```
1 curl https://<aig-ip>:8443/v1/enrollment/status \  
2   -H "Authorization: Bearer 3f2a1b8ec8d...e8c4d9.1749945600"
```

Response 200:

```
1 {  
2   "status": "completed",  
3   "gateway_id": "7708863f-2bb4-42dd-9228-ba41bf8aaa10"  
4 }
```

Deployment

The service listens on a fixed internal port (`8080`). External exposure is controlled entirely by a K8s Service — not by whether the process is running.

```
1 Boot (always):
2   config-api → listens on :8080 (plain HTTP, internal only)
3   No Envoy listener → external traffic cannot reach it
4
5 After operator runs enable-config-api:
6   → Config persisted to K8s Secret (token_hash, expires_at)
7   → Envoy Gateway listener added: :8443 (HTTPS) → config-api:8080
8
9 After operator runs disable-config-api:
10  → Envoy listener removed
11  → Process keeps running; config retained for instant re-enable
```

Security Considerations

- Service binds only to operator-selected interfaces; binding to `0.0.0.0` requires explicit confirmation
- TLS required; plain HTTP disabled
- Failed auth attempts are rate-limited and logged
- Token stored as `sha256(raw_token)` only — never in plaintext
- Raw token shown exactly once at generation time

Open Discussion

1. Standalone service vs. integrated into existing service

The Config API can be implemented as:

- **Standalone process** (`aig-config-api`) — independent lifecycle, easier to isolate and disable; adds another process to manage
- **Integrated into** `aig-dp-mgmt-service` — reuses existing infrastructure (cert handling, K8s access, DMS client); less operational overhead but couples Config API availability to DP health

TBD — requires team discussion before implementation begins.

2. Operator-configurable port

The current design allows the operator to specify the external port via `--port` (default `8443`). An alternative is to fix the port and not expose it as a configurable option.

- **Operator-configurable** — flexible for environments where 8443 is already in use; adds complexity to Envoy listener management since the listener must be created/updated with the

correct port on each enable

- **Fixed port** — simpler implementation; Envoy listener can be pre-created and always present; operator has one less thing to manage

TBD — if most deployments are expected to have port conflicts, configurable is worth the complexity; otherwise fixed is preferred.

3. TLS — share the same cert via Envoy listener

Currently Envoy Gateway handles TLS termination for AI traffic using a K8s TLS Secret (`<gateway-name>`). The Config API can reuse this same cert by adding a second listener to the same Envoy Gateway resource:

```
1 Envoy Gateway (same Gateway resource)
2   Listener: https   port=<ai-traffic-port>  CertificateRefs → <gateway-
   name> Secret      (existing)
3   Listener: config  port=8443                CertificateRefs → <gateway-
   name> Secret      (new)
4
5 HTTPRoute (AI traffic)   → attaches to https listener only
6 HTTPRoute (Config API)  → attaches to config listener → backend:
   config-api:18443
```

Benefits:

- No separate cert lifecycle for Config API
- When operator installs a new cert, Config API automatically picks it up
- Config API service itself serves plain HTTP internally; Envoy handles TLS

Concerns:

- Config API TLS is tied to Envoy availability — if Envoy is down, Config API is also unreachable even if the process is healthy
- If operator wants separate certs for AI traffic vs. management traffic in the future, this would require a refactor